```
import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3
from datetime import datetime

# ── Database ──────────────────────────────────────────────────────────
def init_db():
    conn = sqlite3.connect("jc_prodlimp.db")
    c = conn.cursor()
    c.execute("""
        CREATE TABLE IF NOT EXISTS pedidos (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            cliente TEXT NOT NULL,
            produto TEXT NOT NULL,
            quantidade INTEGER NOT NULL,
            valor_unit REAL NOT NULL,
            total REAL NOT NULL,
            status TEXT NOT NULL DEFAULT 'Pendente',
            data TEXT NOT NULL
        )
    """)
    conn.commit()
    conn.close()

def get_conn():
    return sqlite3.connect("jc_prodlimp.db")

# ── Main App ──────────────────────────────────────────────────────────
class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("JC Prodlimp — Controle de Vendas")
        self.geometry("950x620")
        self.configure(bg="#f0f4f8")
        self.resizable(True, True)
        init_db()
        self._build_ui()
        self.load_pedidos()

    # ── UI ────────────────────────────────────────────────────────────
    def _build_ui(self):
        # Header
        header = tk.Frame(self, bg="#1a56a0", height=60)
        header.pack(fill="x")
```

```
tk.Label(header, text="🧴 JC Prodlimp — Sistema de Vendas",
         bg="#1a56a0", fg="white",
         font=("Arial", 16, "bold")).pack(side="left", padx=20, pady=12)

# Body
body = tk.Frame(self, bg="#f0f4f8")
body.pack(fill="both", expand=True, padx=20, pady=15)

# Left: form
form_frame = tk.LabelFrame(body, text=" Novo Pedido ",
                           bg="#f0f4f8", fg="#1a56a0",
                           font=("Arial", 11, "bold"), bd=2)
form_frame.pack(side="left", fill="y", padx=(0, 15), pady=0, ipadx=10, ip

campos = [("Cliente:", "entry_cliente"),
          ("Produto:", "entry_produto"),
          ("Quantidade:", "entry_qtd"),
          ("Valor unitário (R$):", "entry_valor")]

for i, (label, attr) in enumerate(campos):
    tk.Label(form_frame, text=label, bg="#f0f4f8",
             font=("Arial", 10)).grid(row=i, column=0, sticky="w", pady=6
    entry = tk.Entry(form_frame, width=22, font=("Arial", 10))
    entry.grid(row=i, column=1, pady=6, padx=8)
    setattr(self, attr, entry)

# Status
tk.Label(form_frame, text="Status:", bg="#f0f4f8",
         font=("Arial", 10)).grid(row=4, column=0, sticky="w", pady=6, pa
self.combo_status = ttk.Combobox(form_frame, width=19,
                                 values=["Pendente", "Em andamento", "Ent
                                 state="readonly", font=("Arial", 10))
self.combo_status.set("Pendente")
self.combo_status.grid(row=4, column=1, pady=6, padx=8)

# Buttons
btn_frame = tk.Frame(form_frame, bg="#f0f4f8")
btn_frame.grid(row=5, column=0, columnspan=2, pady=14)

tk.Button(btn_frame, text="➕ Adicionar", command=self.add_pedido,
          bg="#1a56a0", fg="white", font=("Arial", 10, "bold"),
          width=12, cursor="hand2", relief="flat").pack(side="left", padx

tk.Button(btn_frame, text="✏️ Atualizar", command=self.update_pedido,
          bg="#2e75b6", fg="white", font=("Arial", 10, "bold"),
          width=12, cursor="hand2", relief="flat").pack(side="left", padx
```

```
        tk.Button(btn_frame, text="🗑️ Excluir", command=self.delete_pedido,
                  bg="#c0392b", fg="white", font=("Arial", 10, "bold"),
                  width=12, cursor="hand2", relief="flat").pack(side="left", padx

        tk.Button(btn_frame, text="🔄 Limpar", command=self.clear_form,
                  bg="#7f8c8d", fg="white", font=("Arial", 10, "bold"),
                  width=12, cursor="hand2", relief="flat").pack(side="left", padx

        # Total label
        self.lbl_total = tk.Label(form_frame, text="Total: R$ 0,00",
                                  bg="#f0f4f8", fg="#1a56a0",
                                  font=("Arial", 11, "bold"))
        self.lbl_total.grid(row=6, column=0, columnspan=2, pady=6)

        # Right: table
        table_frame = tk.LabelFrame(body, text=" Pedidos ",
                                    bg="#f0f4f8", fg="#1a56a0",
                                    font=("Arial", 11, "bold"), bd=2)
        table_frame.pack(side="left", fill="both", expand=True)

        cols = ("ID", "Cliente", "Produto", "Qtd", "Unit (R$)", "Total (R$)", "St
        self.tree = ttk.Treeview(table_frame, columns=cols, show="headings", heig

        widths = [40, 130, 140, 50, 80, 90, 100, 90]
        for col, w in zip(cols, widths):
            self.tree.heading(col, text=col)
            self.tree.column(col, width=w, anchor="center")

        # Striped rows
        self.tree.tag_configure("even", background="#dce8f7")
        self.tree.tag_configure("odd", background="white")

        scroll = ttk.Scrollbar(table_frame, orient="vertical", command=self.tree.
        self.tree.configure(yscrollcommand=scroll.set)
        self.tree.pack(side="left", fill="both", expand=True, padx=6, pady=6)
        scroll.pack(side="right", fill="y", pady=6)

        self.tree.bind("<<TreeviewSelect>>", self.on_select)

        # Footer
        footer = tk.Frame(self, bg="#1a56a0", height=30)
        footer.pack(fill="x", side="bottom")
        self.lbl_summary = tk.Label(footer, text="", bg="#1a56a0", fg="white",
                                    font=("Arial", 9))
        self.lbl_summary.pack(side="right", padx=16, pady=5)

    # ── CRUD ──────────────────────────────────────────────────────────────
```

```
def add_pedido(self):
    data = self._get_form()
    if not data:
        return
    conn = get_conn()
    conn.execute("""INSERT INTO pedidos (cliente, produto, quantidade,
                    valor_unit, total, status, data)
                    VALUES (?,?,?,?,?,?,?)""", data)
    conn.commit()
    conn.close()
    self.load_pedidos()
    self.clear_form()
    messagebox.showinfo("Sucesso", "Pedido adicionado!")

def update_pedido(self):
    sel = self.tree.selection()
    if not sel:
        messagebox.showwarning("Atenção", "Selecione um pedido para atualizar
        return
    pid = self.tree.item(sel[0])["values"][0]
    data = self._get_form()
    if not data:
        return
    conn = get_conn()
    conn.execute("""UPDATE pedidos SET cliente=?, produto=?, quantidade=?,
                    valor_unit=?, total=?, status=?, data=?
                    WHERE id=?""", (*data, pid))
    conn.commit()
    conn.close()
    self.load_pedidos()
    messagebox.showinfo("Sucesso", "Pedido atualizado!")

def delete_pedido(self):
    sel = self.tree.selection()
    if not sel:
        messagebox.showwarning("Atenção", "Selecione um pedido para excluir."
        return
    pid = self.tree.item(sel[0])["values"][0]
    if not messagebox.askyesno("Confirmar", "Excluir este pedido?"):
        return
    conn = get_conn()
    conn.execute("DELETE FROM pedidos WHERE id=?", (pid,))
    conn.commit()
    conn.close()
    self.load_pedidos()
    self.clear_form()
```

```
def load_pedidos(self):
    for row in self.tree.get_children():
        self.tree.delete(row)
    conn = get_conn()
    rows = conn.execute("SELECT * FROM pedidos ORDER BY id DESC").fetchall()
    conn.close()
    total_geral = 0
    for i, row in enumerate(rows):
        tag = "even" if i % 2 == 0 else "odd"
        self.tree.insert("", "end", values=(
            row[0], row[1], row[2], row[3],
            f"{row[4]:.2f}", f"{row[5]:.2f}", row[6], row[7]
        ), tags=(tag,))
        total_geral += row[5]
    self.lbl_summary.config(
        text=f"Total de pedidos: {len(rows)}  |  Faturamento total: R$ {total
    )

# ── Helpers ──────────────────────────────────────────────────────────
def _get_form(self):
    cliente = self.entry_cliente.get().strip()
    produto = self.entry_produto.get().strip()
    status = self.combo_status.get()
    try:
        qtd = int(self.entry_qtd.get())
        valor = float(self.entry_valor.get().replace(",", "."))
    except ValueError:
        messagebox.showerror("Erro", "Quantidade e valor devem ser números.")
        return None
    if not cliente or not produto:
        messagebox.showerror("Erro", "Preencha cliente e produto.")
        return None
    total = qtd * valor
    self.lbl_total.config(text=f"Total: R$ {total:,.2f}")
    data = datetime.now().strftime("%d/%m/%Y")
    return (cliente, produto, qtd, valor, total, status, data)

def clear_form(self):
    for attr in ("entry_cliente", "entry_produto", "entry_qtd", "entry_valor"
        getattr(self, attr).delete(0, "end")
    self.combo_status.set("Pendente")
    self.lbl_total.config(text="Total: R$ 0,00")
    self.tree.selection_remove(self.tree.selection())

def on_select(self, _):
    sel = self.tree.selection()
    if not sel:
```

```
            return
        vals = self.tree.item(sel[0])["values"]
        self.entry_cliente.delete(0, "end"); self.entry_cliente.insert(0, vals[1])
        self.entry_produto.delete(0, "end"); self.entry_produto.insert(0, vals[2])
        self.entry_qtd.delete(0, "end");    self.entry_qtd.insert(0, vals[3])
        self.entry_valor.delete(0, "end");  self.entry_valor.insert(0, vals[4])
        self.combo_status.set(vals[6])
        try:
            total = int(vals[3]) * float(str(vals[4]).replace(",", "."))
            self.lbl_total.config(text=f"Total: R$ {total:,.2f}")
        except Exception:
            pass

if __name__ == "__main__":
    App().mainloop()
```